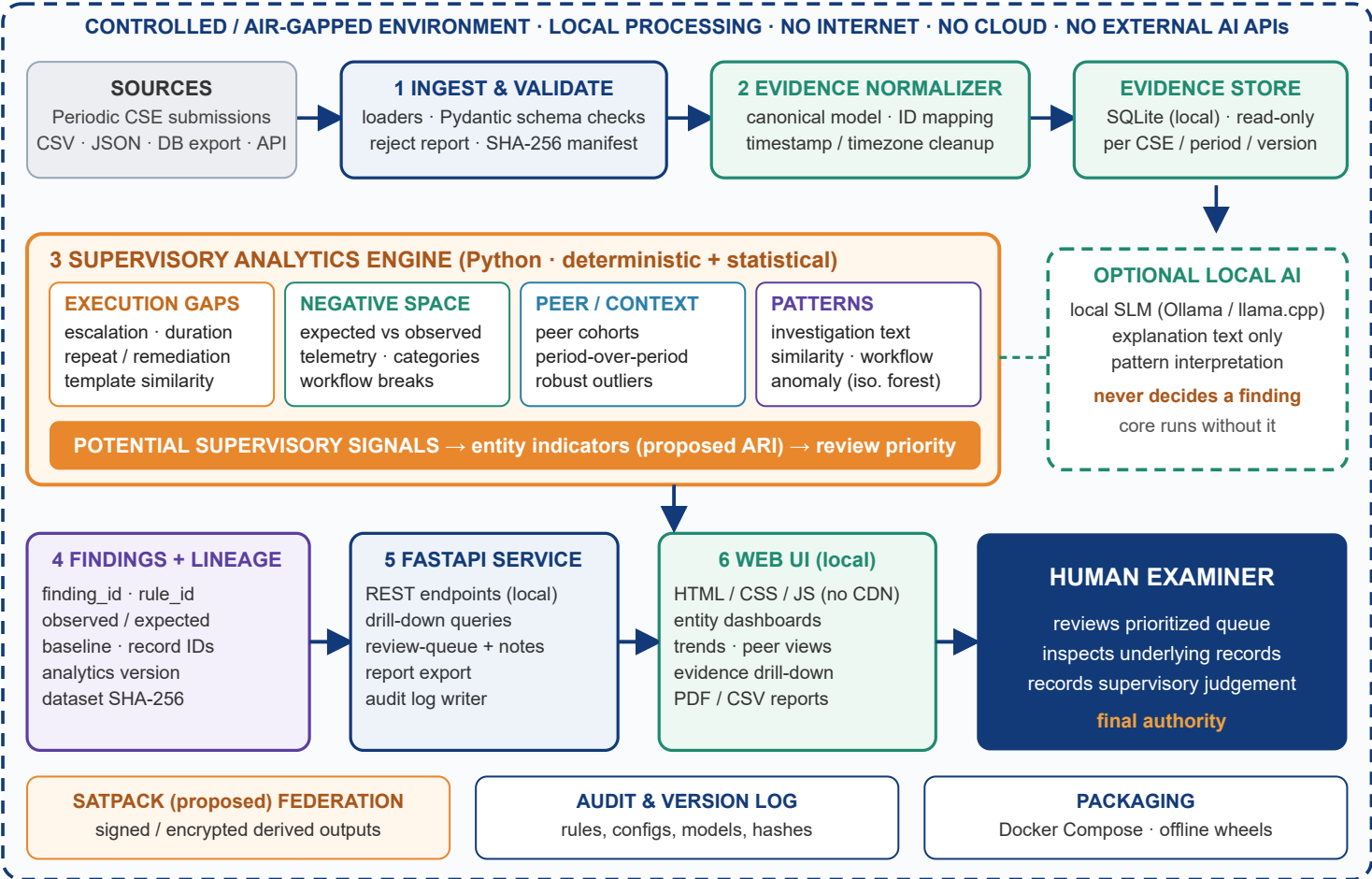


Purpose. AudMi is a supervisory analytics tool, not a SOC, SIEM or monitor. It ingests periodic SOC alert and case-management submissions from multiple CSEs, finds **execution gaps** and **negative space**, explains every finding with traceable evidence, and produces a prioritized review queue for the **human examiner**, who keeps the final judgement.

1 Architecture map



2 Components and technology

Component	Technology (proposed)	Responsibility
Ingest & validate	Python, Pydantic, pandas	Read CSV/JSON/DB exports; check schema, types and timestamps; write a reject report; hash each input file.
Normalizer	Python, SQLite	Map vendor fields to the canonical model (CSE, Asset, Alert, Case, Investigation, Escalation, Remediation); keep source record IDs.
Analytics engine	Python, NumPy, scikit-learn	Versioned rule packs plus statistical baselines; every output is a finding with evidence.
Service layer	FastAPI, Uvicorn	Local-only API for queries, drill-down, review notes and report export.
UI	HTML/CSS/JS, bundled assets	Dashboards, trends, peer comparison, review queue, drill-down to source records.
Optional local AI	Ollama / llama.cpp / vLLM	Drafts plain-language explanations from structured findings. Output is labelled AI-assisted.
Federation	Signed + encrypted archives	Export/import of derived outputs between local installations by controlled offline transfer.

3 Deployment model

- Air-gapped by design:** no internet, cloud, SaaS or external AI API calls. Dependencies ship as an offline bundle (wheels, container images, model files) with checksums.
- Batch, not streaming:** analysis runs per submission and reporting period; no continuous log collection.
- Honest status:** the hosted demo uses synthetic data only. Offline operation is the target architecture and will be claimed only after it is tested on an isolated machine.

4 Analytics methodology

Every module is deterministic or statistical first, so results are reproducible. Each rule has an ID, parameters and a version. Thresholds come from peer and historical baselines (median/MAD per severity and category), not fixed guesses.

Rule ID	Signal	Method
EG-ESC-001	High/critical case, no escalation	Join case → escalation records by severity policy; flag investigation present with escalation absent.
EG-DUR-001	Unusually fast closure	Closure time vs peer/severity baseline using robust z-score; also checks time between open, first action and close.
EG-REP-001	Repeat alerts, weak remediation	Same asset + category within a window with no remediation or root-cause record between occurrences.
EG-TPL-001	Repetitive investigations	Near-duplicate notes and step sequences (TF-IDF cosine or MinHash) within and across analysts and cases.
NS-TEL-001	Silent critical asset	Expected activity from asset criticality, type and peer assets; flag assets well below that expectation.
NS-CAT-001	Absent alert category	Compare category mix against peer cohort and the entity's own history; flag expected categories that never appear.
NS-WFL-001	Broken workflow chain	Alert without case, case without investigation, closed case without disposition; expected chain vs observed chain.
CX-PEER-001	Peer and period deviation	Percentile rank in peer cohort; period-over-period change; Isolation Forest only as a secondary outlier flag.

How "expected" is defined: expectation = asset/control context + historical and peer baseline. Absence counts as a signal only when that baseline says the evidence should exist.

Entity indicator (ARI, proposed): normalized signal scores combined with configurable weights; the breakdown is always shown and the score is a sorting aid, not a verdict. **Output of the engine:** entity, process and sample priority, built from the same findings.

5 Evidence lineage and explainability

Finding record (stored for every signal):

```
finding_id · rule_id · rule_version
entity · period · severity_of_signal
observed · expected · baseline
source_record_ids[] · dataset_sha256
created_at · analytics_version
rationale_text (template-generated)
```

- Chain:** signal → rule → observed pattern → source records → rationale → examiner.
- Traceability:** UI drill-down resolves every finding to the exact input rows.
- Auditability:** append-only audit log; SHA-256 hashes give tamper-evidence, not proof that source data is true or complete.
- Language:** "potential supervisory signal", never "violation".

6 Data requirements

Dataset	Minimum fields
Alerts	alert_id, timestamp, severity, category, asset_id, source/rule name
Cases	case_id, linked alert_ids, open/close time, status, disposition, pseudonymized analyst ID
Investigation	case_id, step/action timestamps, short note text (optional but improves pattern analysis)
Escalation	case_id, escalation time, level, recipient role
Assets (if available)	asset_id, criticality, type, environment; enables negative-space analysis

No raw logs, packet captures, customer data or personal data are needed. Missing optional fields lower the confidence of a signal and are reported, not guessed.

7 Validation methodology

- Synthetic golden dataset:** generated with seeded execution gaps and negative-space conditions plus ground-truth labels, across several CSEs and periods.
- Measures:** precision, recall, false positives, detection coverage, and sampling efficiency (seeded issues found per examiner-hour compared with random sampling). Results are reported only after running them.
- Robustness:** optional public/proxy datasets test behaviour on data AudMi did not generate. They do not represent NCIIPC or real CSE data.
- Expert review:** final validation compares AudMi's queue with findings from expert manual review of the same sample, as the PS requires.

8 Infrastructure and update approach

Item	Planning assumption (to be confirmed by testing)
Core system	Single Linux server or workstation, CPU only, multi-core, 16–32 GB RAM, SSD; Docker Compose; no GPU required.
Optional local SLM	Quantized small model; more RAM or a single GPU helps speed. Loaded offline from a checksummed model file; inference stays local.
Updates	Signed offline bundles carry new rule packs, baselines and model files. Checksums are verified on import and versions are written into every finding.
Scale	Batch jobs run per entity and period in parallel; scaling figures will be measured on the prototype, not assumed.